

Inhabited Parsing

A certified parser for TOML

Synthesized from a *declarative grammar*, verified in **Rocq (Coq)** — surface parser **and** the document state machine

`theories/Toml.v` · S8 — key/value pairs, dotted keys, tables, arrays-of-tables

The core idea

The **innovation**: the *implementation* and the *certification proof* can both be produced by **AI** — from the **BNF** *plus explicit disambiguation decisions*.

grammar (declarative) + ambiguity policy $\Rightarrow$ parser (executable) $\Rightarrow$ proof (certificate)

The BNF is *relational* — `Alt` is unordered, `Many` admits the empty branch, and whitespace runs can be split arbitrarily. The parser additionally commits to *ordered choice* and a declared *separator policy* — those are separate declarative decisions, and TOML's surface makes exactly this point concretely.

TOML grammar — high-level BNF (S8 slice)

```
document ::= ws stmt ( ws1 stmt )* ws

stmt      ::= kv | table | array
kv        ::= key ws "=" ws int           -- key = value (integer-only)
table     ::= "[" key "]"                 -- [table]
array     ::= "[" key "]"                 -- [[array-of-tables]]

key       ::= ident ( "." ident )*        -- bare / dotted, no quoting
ident     ::= ident-char+                 -- any non-ws, non-structural char
int       ::= digit+                     -- non-negative, maximal munch

ws        ::= ( space | tab | lf | cr )*   -- zero or more
ws1       ::= ( space | tab | lf | cr )+   -- one or more ← the separator
```

`ws1` (not `ws0`) between statements is a **declared disambiguation decision**, not part of the relational BNF — it is what makes the greedy parser agree with the relation.

The pipeline



TOML adds a **second layer**: the document *state machine* (`step` / `run`) is itself a declarative spec — `Get` / `Put` / `Guard` over a `Namespace` .

Types at a glance — grammar & meaning

```
Spec  $\Gamma$  N A
Grammar  $\Gamma$  N := forall A, N A -> Spec  $\Gamma$  N A
denote G w S  $\gamma$  i a  $\gamma'$  j
```

(* grammar: a combinator tree, as data *)
(* productions table; recursion via Call *)
(* "S, at i in state γ , yields a at j in γ' " *)

Spec parameter	what it is	surface layer	state layer
Token	token type (section var)	ascii	unit
Γ	environment / state	unit (stateless)	Namespace
N	nonterminal family	tom_nt (empty — no recursion)	tom_nt
A	result type produced	stmt / Document	unit

``denote`` is **`Type`-valued** — a derivation is the certificate: the parsed value ``a`` is an index of the proof. Two grammars over the same ``tom_nt``: `surface_grammar : Grammar ascii unit tom_nt` and `tom_grammar : Grammar unit Namespace tom_nt`.

Types at a glance — parser & data

```
parse_doc (fuel : nat) (w : list ascii) : option Document
parse_then_validate (w : list ascii) : option Namespace
direct_parse (fuel : nat) (ns : Namespace) (w : list ascii) : option Namespace
```

```
Definition seg : Type := list ascii.          (* a key segment *)
Definition key : Type := list seg.             (* a dotted path  *)
Definition Namespace : Type := list (key * kind).
```

```
Inductive kind : Type := KImplicit | KTable | KArrayTable | KScalar.
Inductive stmt : Type :=
| SKV (k : key) (v : nat) | STable (k : key) | SArray (k : key).
Definition Document : Type := list stmt.
```

The surface parser reads characters $\rightarrow$ `Document`. The state machine folds `step` over it: `run ns (s :: rest) = step ns s >>= fun ns'  $\rightarrow$  run ns' rest`. `parse_then_validate = parse_doc  $\circ$  run []`; `direct_parse` interleaves the two (single pass, §6.5), certified equivalent to parse-then-validate.

Step 1 — grammar as data

The `Spec` deep embedding; recursion via `Call` + a `Grammar` .

```
Inductive Spec (Γ : Type) (N : Type -> Type) : Type -> Type :=
| Pure   : A -> Spec Γ N A
| Fail   : Spec Γ N A
| Tok     : (Token -> Type) -> Spec Γ N Token
| Seq     : Spec Γ N A -> Spec Γ N B -> Spec Γ N (A * B)
| Alt     : Spec Γ N A -> Spec Γ N A -> Spec Γ N A
| Map     : (A -> B) -> Spec Γ N A -> Spec Γ N B
| Bind    : Spec Γ N A -> (A -> Spec Γ N B) -> Spec Γ N B
| Many   : Spec Γ N A -> Spec Γ N (list A)
| Guard   : (A -> Type) -> Spec Γ N A -> Spec Γ N A
| Get | Put γ | Local | Call (n : N A).
```

Semantic actions live in `Map` / `Bind` ; `Get` / `Put` / `Guard` express the *state machine* (the second layer). A derivation is a derivation-indexed value.

Step 1 — the surface productions, concretely

```
Definition ch c := Map (fun _ : ascii => tt) (Tok (fun c' => c' = c)).
```

```
Definition ws_spec := Map (fun _ : list unit => tt) (Many (Map (fun _ => tt) (Tok is_ws))).
```

```
Definition ws1_spec := Map (fun _ => tt) (Seq (Map (fun _ => tt) (Tok is_ws))  
      (Many (Map (fun _ => tt) (Tok is_ws)))).
```

```
Definition int_spec := Map (fun p => digits_to_nat (fst p :: snd p) 0)  
      (Seq digit_spec (Many digit_spec)).
```

```
Definition ident_spec := Map (fun p => fst p :: snd p)  
      (Seq (Tok is_ident_char) (Many (Tok is_ident_char))).
```

```
Definition key_spec := Map (fun p => fst p :: snd p)  
      (Seq ident_spec (Many (Map (fun p => snd p)  
      (Seq (ch ".") ident_spec))))).
```

``ws1_spec` = `ws · ws*` — one-or-more. `int_spec` = `digit digit*`. `key_spec` = `ident (.ident)*`.`

Step 1 — statements & the document

```
Definition kv_spec := Bind key_spec (fun k =>
  Bind ws_spec (fun _ => Bind (ch "=") (fun _ =>
    Bind ws_spec (fun _ => Map (fun v : nat => SKV k v) int_spec))))).

Definition table_spec := Bind (ch "[") (fun _ =>
  Bind key_spec (fun k => Bind (ch "]") (fun _ => Pure (STable k)))).

Definition array_spec := Bind (ch "[") (fun _ => Bind (ch "[") (fun _ =>
  Bind key_spec (fun k => Bind (ch "]") (fun _ =>
    Bind (ch "]") (fun _ => Pure (SArray k)))))).

Definition stmt_spec := Alt kv_spec (Alt table_spec array_spec).

Definition doc_spec := Bind ws_spec (fun _ =>
  Bind (Alt (Pure []) (Map (fun p => fst p :: snd p)
    (Seq stmt_spec (Many (Bind ws1_spec (fun _ => stmt_spec))))))
  (fun doc => Bind ws_spec (fun _ => Pure doc))).
```

$\text{doc_spec} = \text{ws} \cdot (\epsilon \mid \text{stmt} (\text{ws1 stmt})^*) \cdot \text{ws}$. The separator is ws1_spec — the disambiguation.

The declarative disambiguation choices

The BNF is relational; the parser commits to these **separate**, **declared** decisions:

decision	what it pins down	why it matters
ordered choice	<code>stmt = kv table array ,</code> tried in order	first char <code>[</code> disambiguates table/array from <code>kv</code> ; <code>[[</code> picks array
separator = <code>ws1</code>	statements split on ≥ 1 whitespace, not <code>ws0</code>	forces maximal-munch on the trailing integer — the crux
maximal munch	<code>int = digit+ , greedy</code>	<code>12</code> parses as one value, never <code>1</code> then <code>2</code>
bare keys only	<code>ident-char = ¬(ws v "=" v "["</code> <code>v "]" v ".")</code>	no quoted keys in this slice
no Call recursion	<code>tom_nt</code> is empty	a document is a <i>flat</i> list; repetition is <code>Many</code> , not nesting

Why `ws1` (not `ws0`) is the crux

With `ws0` separators, the relation is **more permissive than the greedy parser**:

```
a=12b=3    denotable as [SKV a 12 ; SKV b 3]    (greedy, correct)
           denotable as [SKV a 1  ; SKV 2b 3]    (relation also admits this)
a=12=3      denotable as [SKV a 1  ; SKV 2 3]    but parse_doc = None
```

- Many `digit_spec` is *non-maximal*: `12` can denote as `1` then `2`. With `ws0`, a following statement can absorb the leftover digit — so `denote` $\Rightarrow$ `parse` (**completeness**) fails.
- With `ws1`: a statement *must* be followed by at least one whitespace before the next, so the integer inside a `kv` is forced to consume **all** its digits. Maximal-munch becomes a *consequence of the separator*, and `doc_complete` holds.

This is the headline result: **choosing `'ws1'` is what makes the grammar unambiguous, not a change to the parser.**

Step 2 — the denotation

`denote` is an **inductive relation** — a derivation *is* the certificate.

```
Inductive denote : forall A, Spec  $\Gamma$  N A ->  $\Gamma$  -> nat -> A ->  $\Gamma$  -> nat -> Type :=
| d_pure : denote (Pure a)  $\gamma$  i a  $\gamma$  i
| d_tok   : nth_error w i = Some t -> P t -> denote (Tok P)  $\gamma$  i t  $\gamma$  (S i)
| d_seq | d_alt_l | d_alt_r | d_map | d_bind | d_guard
| d_many_nil | d_many_cons
| d_get | d_put | d_local
| d_call : denote (G A n)  $\gamma$  i a  $\gamma'$  j -> denote (Call n)  $\gamma$  i a  $\gamma'$  j.

(* denote G w S  $\gamma$  i a  $\gamma'$  j  :=  "S, begun at i in env  $\gamma$ ,
   produces a and ends at j in env  $\gamma'$ "    - proof-relevant (Type) *)
```

``d_many_cons`` *chooses* a split; the relation admits every split. Completeness must show the greedy split is the one the parser picks — hence the ``ws1`` argument.

Step 3 — the parser

A fuel-bounded recursive-descent `Fixpoint` (first-character dispatch), plus the state machine.

```
Fixpoint parse_doc (fuel : nat) (w : list ascii) : option Document :=
  match skip_ws w with
  | [] => Some []
  | _ => match fuel with
    | 0 => None
    | S fuel' =>
      match parse_stmt fuel' (skip_ws w) with
      | None => None
      | Some (s, rest) =>
        match skip_ws rest with
        | [] => Some [s]
        | _ => match rest with
          | c :: _ => if is_ws c then parse_doc fuel' rest ↪ cons s
            else None
          _
        end
      end
    end
  end
end
```

```
Fixpoint run (ns : Namespace) (doc : Document) : option Namespace :=
  match doc with [] => Some ns | s :: rest => step ns s >>= fun ns' => run ns' rest end.
```

Step 4 — soundness ✓

The parser only ever produces real denotations.

```
Lemma parse_doc_tail_sound : (* the Many (ws1 · stmt) tail *)
  parse_doc fuel w = Some ss -> ws_part w <> [] ->
  denote surface_grammar (prefix ++ w) (Many (Bind ws1_spec (fun _ => stmt_spec)))
  tt (length prefix) ss tt (length prefix + length w - length (ws_trail w)).

Lemma parse_doc_body_sound : (* Alt (Pure []) (Map (Seq stmt (Many ...))) *)
  parse_doc fuel w = Some ss ->
  denote surface_grammar (prefix ++ skip_ws w) (Alt (Pure []) (Map ...))
  tt (length prefix) ss tt (length prefix + length (skip_ws w) - length (ws_trail (skip_ws w))).

Lemma parse_doc_sound : forall fuel w doc,
  parse_doc fuel w = Some doc -> denote surface_grammar w doc_spec tt 0 doc tt (length w).
```

Status: **all `Qed.`** — key/value/table/array/statement/document, plus the state machine (`step_spec_sound` , `direct_parse_equiv` , `parse_then_validate_iff_direct`).

Step 5 — completeness (the subtlety)

The converse is **not** free — it holds *because* of the `ws1` separator.

```
Lemma key_complete : (* key: maximal ident, then '.' / '=' dispatch *)  
  denote ... key_spec ... -> (non-ident, non-dot after) -> parse_key ... = Some (k, suffix).
```

```
Lemma stmt_complete : (* statement: maximal integer *)  
  denote ... stmt_spec ... -> (non-digit after) -> parse_stmt ... = Some (s, suffix).
```

```
Lemma doc_complete : forall w doc,  
  denote surface_grammar w doc_spec tt 0 doc tt (length w) ->  
  parse_doc (length w) w = Some doc.
```

- `stmt_complete` demands maximality — “the char after the statement is a non-digit”. It is supplied by the `ws1_spec` that follows: a separator must begin with whitespace, so the integer’s last digit cannot be followed by another digit.
- `doc_complete` is proved by induction on the document, reconstructing `Many (ws1 · stmt)` step by step and threading the maximality witness.

Step 6 — extraction & acceptance

```
Extraction Language OCaml.
```

```
Extraction "toml.ml" parse_then_validate direct_parse parse_doc.
```

```
parse_then_validate "title = 42"           = Some [("title", KScalar)]
parse_then_validate "a.b = 7"              = Some [("a.b", KScalar); ("a", KImplicit)]
parse_then_validate "[t]\nid = 1"          = Some [(id, KScalar); (t, KTable)]
parse_then_validate "[[a]]\nitem = 1"      = Some [(item, KScalar); (a, KArrayTable)]
```

Build: `nix develop --command make` (flake pins Rocq 9.1.1). Extracted types are clean: `stmt = SKV of key × Z.t | STable of key | SArray of key`; proofs vanish.

Benchmark — extraction directives

```
Extract Inductive nat    => Z.t      (* Big_int_Z.big_int = Zarith Z.t *)
Extract Inductive Z      => Z.t
Extract Inductive ascii => char
Extract Constant Z.of_nat => "(fun n -> n)"  (* else it recurses on the value *)
```

``nat`/`Z`` $\rightarrow$ Zarith ``Z.t`` (arbitrary precision), ``ascii`` $\rightarrow$ ``char``. A hand-written ``Big_int_Z.ml`` shim supplies the ``Big_int``-named API over ``Z.t``.

Benchmark — surface parser scales linearly

`parse_doc` (surface parser only) on a synthetic flat document (`key{i} = v + grp{i}.leaf{i} = v` per item):

items	size	statements	time	throughput
2k	86 KB	4.0k	0.005 s	~18 MB/s
4k	176 KB	8.1k	0.006 s	~28 MB/s
6k	267 KB	12.1k	0.009 s	~30 MB/s
80k	3.9 MB	162k	0.155 s	~25 MB/s
160k	8.1 MB	323k	0.416 s	~19 MB/s

Linear, on par with the JSON parser (~30 MB/s). The tail-off is ``big_int`` fuel arithmetic and cache pressure, not algorithmic growth.

Benchmark — the state machine is $O(n^2)$

`parse_then_validate` / `direct_parse` fold the **flat-namespace** state machine, whose `lookup : list (key × kind) → key → option kind` is a **linear scan** of a growing list.

items	size	<code>parse_doc</code>	<code>parse_then_validate</code>	<code>direct_parse</code>
2k	86 KB	0.005 s	0.200 s	0.199 s
4k	176 KB	0.006 s	0.778 s	0.807 s
6k	267 KB	0.009 s	2.46 s	2.99 s

- Each `step` re-scans the namespace → **quadratic** in the number of statements (4× time per 2× input).
- This is the *naive certified* implementation, not an extraction artifact. Optimizing `lookup` to a map would change `run` / `step` and require re-proving `DocumentStep` — out of scope for S8.

Thanks

Soundness certified · **Completeness** certified (the `ws1` fix) · **Extraction** to OCaml

```
./slides/toml — npm install && npm run dev
```